the main PID hierarchy. A process inside a PID namespace can have the same PID as a process outside it, and even inside the namespace, you can have different *init* with PID 1.

UTS namespace: In the UTS (UNIX Timesharing System) namespace, a process can have a different set of domain names and host names than the main system. It uses *sethostname()* and *setdomainname()* to do that.

IPC namespace: This is used for inter-process communication resources isolation and POSIX message queues.

User namespace: This isolates user and group IDs inside a namespace, which is allowed to have the same UID or GID in the namespace as in the host machine. In your system, unprivileged processes can create user namespaces in which they have full privileges.

Network namespace: Inside this namespace, processes can have different network stacks, i.e., different network devices, IP addresses, routing tables, etc.

Sandboxing tools available in Linux use this namespaces feature to isolate a process or create a new virtual environment. A much more secure tool will be that which uses maximum namespaces for isolation. Now, let's talk about different methods of sandboxing, from soft to hard isolation.

chroot

chroot is the oldest sandboxing tool available in Linux. Its work is the same as mount namespace, but it is implemented much earlier. *chroot* changes the root directory for a process to any *chroot* directory (like */chroot*). As the root directory is the top of the file system hierarchy, applications are unable to access directories higher up than the root directory, and so are isolated from the rest of the system. This prevents applications inside the *chroot* from interfering with files elsewhere on your computer. To create an isolated environment in old SystemV based operating systems, you first need to copy all required packages and libraries to that directory. For demonstration purposes, I am running *'ls'* on the *chroot* directory.

First, create a directory to set as root a file system for a process:

```
#mkdir /chroot
```

Next, make the required directory inside it.

```
#mkdir /chroot/{lib,lib64,bin,etc}
```

Now, the most important step is to copy the executable and libraries. To get the shell inside the *chroot*, you also need */bin/bash*.

```
#cp -v /bin/{bash,ls} /chroot/bin
```

To see the libraries required for this script, run the following command:

```
#ldd /bin/bash
linux-vdso.so.1 (0x00007fff70deb000)
libncurses.so.5 => /lib/x86_64-linux-gnu/libncurses.so.5
(0x00007f25e33a9000)
libtinfo.so.5 => /lib/x86_64-linux-gnu/libtinfo.so.5
(0x00007f25e317f000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2
(0x00007f25e2f7a000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
(0x00007f25e2bd6000)
/lib64/ld-linux-x86-64.so.2 (0x00007f25e360d000)

#ldd /bin/ls
linux-vdso.so.1 (0x00007fff4f8e6000)
libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1
(0x00007f9f08aec000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
(0x00007f9f08748000)
libpcre.so.3 => /lib/x86_64-linux-gnu/libpcre.so.3
(0x00007f9f004d7000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2
(0x00007f9f002d3000)
/lib64/ld-linux-x86-64.so.2 (0x00007f9f00d4f000)
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0
(0x00007f9f000b6000)
```

Now, copy these files to the *lib* or *lib64* of */chroot* as required.

Once you have copied all the necessary files, it's time to enter the *chroot*.

```
#sudo chroot /chroot/ /bin/bash
```

You will be prompted with a shell running inside your virtual environment. Here, you don't have much to run besides *ls*, but it has changed the root file system for this process to */chroot*.

To get a more full-featured environment you can use the *debootstrap* utility to bootstrap a basic Debian system:

```
#debootstrap --arch=amd64 unstable my_deb/
```

It will download a minimal system to run under *chroot*. You can use this to even test 32-bit applications on 64-bit systems or for testing your program before installation. To get process management, mount *proc* to the *chroot*, and to make the contents of *home* 'lost on exit', mount *tmpfs* at */home//*:

```
#sudo mount -o bind /proc my_deb/proc
#mount -t tmpfs -o size=100m tmpfs /home/user
```